

Lesson: Imports as Systems (Effects, Dependencies, and Design)

Every import you write creates a **connection between parts of your system**.

That connection has:

- **effects** (what happens when something changes)
- **points of failure** (where things can break)

1. Imports = Dependencies

When you write:

`</> Python`

```
from mylib.math_utils import add
```

You are saying:

"This file depends on `math_utils` to function."

System Insight

If `math_utils` breaks → everything importing it can break.

This is called a **dependency chain**.

Example System:

```
main.py
  ↓
math_utils.py
  ↓
helper.py
```

If `helper.py` fails, the failure travels upward.

2. Ripple Effects (Change Propagation)

Example:

You change:

```
</> Python  
  
def add(a, b):
```

to:

```
</> Python  
  
def add(a, b, c):
```

Now:

- Every file importing `add` breaks
- This is a **breaking change**

💡 Lesson

Your imports define how **sensitive your system is to change**

3. 🎯 Points of Failure

A **point of failure** is a place where one issue can break many things.

Common Import-Based Failure Points

1. Central Utility File

```
utils.py
```

Everything imports from it.

Problem:

- If it breaks → entire system breaks

2. Deep Dependency Chains

```
A → B → C → D
```

If D fails:

- A, B, C all fail indirectly

3. Circular Imports

```
A imports B  
B imports A
```

Result:

- Python crashes or behaves unpredictably

4. Coupling (Critical Concept)

Coupling = how tightly connected your files are

High Coupling ❌

Python

```
from mylib.math_utils import add, multiply, divide, mod, complex_func
```

Problems:

- Hard to change anything
- Everything depends on everything

Low Coupling ✅

Python

```
from mylib import add
```

Better because:

- You control exposure via `__init__.py`
- Internal changes don't break users

5. 🧱 Abstraction Layers (How Pros Design Systems)

Instead of exposing everything directly:

```
main.py → mylib → internal modules
```

Example:

```
mylib/__init__.py
```

</> Python

```
from .math_utils import add
```

Internal structure:

```
mylib/  
  math_utils.py  
  helpers.py
```

💡 Effect

- You can change `helpers.py` freely
- `main.py` is unaffected

🧠 This is called:

Encapsulation

6. ⚠️ Dangerous Import Patterns

1. Wildcard Imports ❌

</> Python

```
from module import *
```

Effects:

- Namespace pollution
- Hidden dependencies
- Hard debugging

2. Deep Imports ❌

</> Python

```
from mylib.utils.helpers.math.internal import add
```

Effects:

- Breaks if structure changes
- Too tightly coupled

3. Importing Everything ❌

</> Python

```
import massive_module
```

When you only need one function.

Effects:

- Slower startup
- Unnecessary dependencies

7. ✅ Strong Import Practices

1. Design a Public API

Only expose what users need:

`</>` Python

```
# mylib/__init__.py
from .math_utils import add
```

2. Keep Internal Logic Hidden

Users should NOT rely on:

`</>` Python

```
mylib.helpers.internal_function
```

8. 🧠 System Stability Strategy

Think in layers:

```
Layer 1 (Stable API) → rarely changes
Layer 2 (Core Logic) → changes sometimes
Layer 3 (Helpers) → changes often
```

9. 🔄 Import Direction Rule

Dependencies should flow **one way**:

```
main → api → core → helpers
```

✗ Never:

```
helpers → main
```

10. 🧪 Real-World Thinking

When you import something, ask:

- If this changes, what breaks?
- Am I importing too deep?
- Is this a stable function?
- Am I exposing too much?

11. 📌 Do's and Don'ts

✓ **DO:**

- Use `__init__.py` to control access
- Keep imports simple and shallow
- Group related functionality
- Design for change

✗ DON'T:

- Use `import *`
- Create circular imports
- Expose internal functions
- Build long dependency chains
- Name files after built-ins

12. 🧠 Final Mental Model

Your import system is like a **power grid**:

- Modules = power stations

- Imports = power lines
- Failures spread through connections

Good design:

- isolates failures
- controls flow
- keeps system stable